UTM

UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SECP3843-01 SPECIAL TOPIC IN DATA ENGINEERING

SEMESTER 6

2025/2026

TITLE: TUTORIAL 3 AI ALGORTHM

LECTURER: DR. ARYATI BINTI BAKRI

NAME	MATRIC NO
NABIL AFLAH BOO BINTI MOHD YOSUF BOO YONG CHONG	A23CS0252
HARINI A/P SANGARAN	A23CS0081

TABLE OF CONTENT

a. What is Image Classification?

The classification of an image is one of the most fundamental tasks of computer vision in which a given image is assigned a label or category that is defined beforehand. Image classification is a classification that makes a single prediction about the content of an image, as opposed to object detection, which detects and locates multiple objects in an image. All images of the same class should be grouped together such that all the images of cars driving down the road should be in the same class as "Car". The model receives an image and it will go through a sequence of layers of computation that gradually learn to notice meaningful features in the image, like edges, textures, colors or shapes. These learned features are fed into the decision part to classify the image into one of the available class labels using the highest probability score for the prediction.

b. What is CNN?

Convolutional Neural Network (CNN) or ConvNet is a part of machine learning that counts as a subset of the many artificial neural network (ANN) models used for different aims and datasets. CNNs are made to handle structured grid data like images, so that they can fit image classification nicely. As in the video, a CNN takes in an image of a cube for instance, and runs it through several hidden layers that are made up of linked neurons, and it produces an output guess in the last layer. Those hidden layers do the heavy lifting by learning how to pull out more nuanced features, starting with very simple cues like edges then moving toward higher level ideas such as shapes and objects. All together, the input layer, hidden layer, and output layer help the CNN automatically discover what matters most from raw image data. Thus, there is no need for manual feature engineering, which made CNN become a very effective and strong model for image classification.

c. Evaluation of CNN

Evaluating a CNN is an important part of the process when deciding whether or not the CNN has learned to classify the images accurately when it encounters data it has never seen before. This evaluation process is generally performed after training, on a separate test set of data, which the model has never seen while it was being trained. It thus guarantees that the model's performance is not due to memorizing training examples, but to its actual generalization capacity.

Among the most frequently used evaluation metrics in image classification tasks, there is the accuracy, which is the fraction of correctly classified images in the test set. Although accuracy is a simple and straightforward assessment of overall performance, it may not be enough in practice, especially in the presence of imbalance distribution of data across the classes. In these situations, other evaluation parameters like precision, recall, and the F1 score provide a more balanced measurement of the model's accuracy on the individual classes.

The confusion matrix is another useful evaluation tool that gives a detailed breakdown of the correct predictions and incorrect predictions for each class in CNN. It provides a way for practitioners to determine which classes are the most commonly misclassified by the model, thus providing important information about the model's difficulty and informing continued development.

The performance of the model is also tracked during training by providing the training and validation loss as well as accuracy curves over each epoch. A good model would be expected to lose fewer elements over time and become more accurate over time for both the training set and the validation set. When accuracy in the training set shows steady improvement but in the validation set it shows a steady decline, or the same level of accuracy, it's an indication that the model is becoming overfitted or over trained. If the accuracy is still low, both training and validation set will be an underfitting model, meaning that the model has not learnt enough patterns of the data.

d. Steps hand on in Python

The Canadian Institute for Artificial Intelligence Repository (CIFAR-10) dataset is a well-known computer vision benchmark dataset that is used in the implementation of image classification using CNN in a Python environment. CIFAR-10 is a collection of 60,000 color images of 10 mutually exclusive classes including airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck, each of them are 32x32 pixels in size. The dataset is divided into 10,000 testing images and 50,000 training images. As these two APIs are high level and easier to use to create and train deep learning models, the implementation is carried out with TensorFlow and Keras.

First, all of the necessary libraries and modules need to be imported. This comprises TensorFlow and Keras to create a model, NumPy for numerical operations, and Matplotlib as

a tool for visualizing images and the outcomes of training. These are the libraries to support the implementation process overall.

```
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
import numpy as np
import matplotlib.pyplot as plt
```

Figure 1 : Importing Libraries

The second step is to load and explore the dataset. The CIFAR-10 dataset is included in the Keras API as a preload dataset and has been automatically divided into train and test sets. The images and labels are represented by two different arrays. A subset of the data is then displayed in order to get a feel for the data prior to processing.

```
(X_train, y_train), (X_test, y_test) = datasets.cifar10.load_data()
```

Figure 2 : Loading and Exploring Dataset

Third is data preprocessing where the images need to be preprocessed before being input to the model. The pixel values are scaled from 0 to 255 in each image, and then divided by 255.0 so that the values are in the range 0 to 1. This normalization technique will make the model converge faster and smoothly since it will not be influenced by any pixel's value.

```
X_train = X_train / 255.0
X_test = X_test / 255.0
```

Figure 3 : Data Preprocessing

Next, the CNN model architecture would be built. The CNN model is built using the sequential API of Keras where the layers can be stacked sequentially. The architecture is a combination of convolutional and max pooling layers. Then, comes the second convolutional layer having 64 number of filters and another max pooling layer. To extract deeper features, a third convolutional layer having 64 filters is added. The output is then converted to a one dimensional vector of 64 neurons after which it is fed into a fully connected dense layer with 10 neurons associated with the 10 classes of the dataset.

```
cnn = models.Sequential([
    layers.Conv2D(filters=32, kernel_size=(3, 3), activation='relu', input_shape=(32, 32, 3)),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(filters=64, kernel_size=(3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),

    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])
```

Figure 4 : Building CNN Model Architecture

The following step is to compile the model. Once the architecture has been chosen, the model must be compiled. The process includes defining the optimizer, loss function and the evaluation metric. Adam optimizer is the best choice as it is efficient and adaptive learning rate. Labels are not one-hot encoded vectors, so the loss function used is the Sparse Categorical Crossentropy loss. The main measure of evaluation is accuracy.

```
cnn.compile(optimizer='adam',
            loss = 'sparse_categorical_crossentropy',
            metrics=['accuracy'])
```

Figure 5 : Compiling the Model

Step 6 is the model training. The training dataset is used to train the model for a given number of epochs. The model trains by continually updating its weight, reducing the loss over and over until it starts to make better predictions. Part of the training data is reserved as a validation dataset to help the model be tested at the end of each epoch of training with previously unseen examples.

```
cnn.fit(X_train, y_train, epochs=10)
```

Figure 6 : Training the Model

The model is evaluated at last. After training, the model is evaluated on the test set for assessing the generalization ability. The test accuracy is printed to showcase the performance of the model on images which it never saw before. Moreover, the train and validation accuracy, and loss curves are shown against epochs to obtain a visual understanding of overfitting and underfitting.

```
cnn.evaluate(X_test, y_test)
```

Figure 7 : Evaluating the Model

e. What is the weakest in the current CNN model?

The current CNN model has higher training accuracy than the test accuracy, which means that the current CNN model is experiencing overfitting because it memorizes the training data but doesn't generalize well. The model also has limited architecture which has only a few convolutional layers, limiting its ability to learn complex features. Thirdly, the CNN model has no regularization techniques because the model does not use Dropout, Batch Normalization or Data Augmentation, making it prone to overfitting.

f. How to enhance the CNN model?

The current CNN model can be improvised by adding convolutional layers which will extract more complex features. Moreover, batch normalization can be introduced where it normalizes activations between layers, stabilizing and speeding up training. Dropout layers randomly drop neurons during training to reduce overfitting. Lastly, data augmentation can be introduced where techniques like flipping, rotation, and zooming to artificially expand the training dataset and improve generalization.

g. What is the evaluation based on CNN and the new_CNN model?

The training accuracy for both CNN and the new_CNN model are high. However, the test accuracy for the new_CNN model is significantly higher than the CNN model. The new_CNN model shows better generalization, with the training and testing accuracy curves being much closer together, showing reduced overfitting.

h. Results and Discussion

The CIFAR-10 dataset was successfully loaded and processed using TensorFlow and Keras. During the exploratory stage, sample images from the dataset were visualized to verify that the images and labels were correctly loaded. Figure 8 shows the output for the sample images from the dataset.

```
[7]: classes = ["airplane","automobile","bird",
                "cat","deer","dog",
                "frog","horse","ship","truck"]

     plt.figure(figsize=(10,5))

     for i in range(9):
         plt.subplot(3,3,i+1)
         plt.imshow(X_train[i])
         plt.title(classes[y_train[i][0]])
         plt.axis("off")

     plt.show()
```

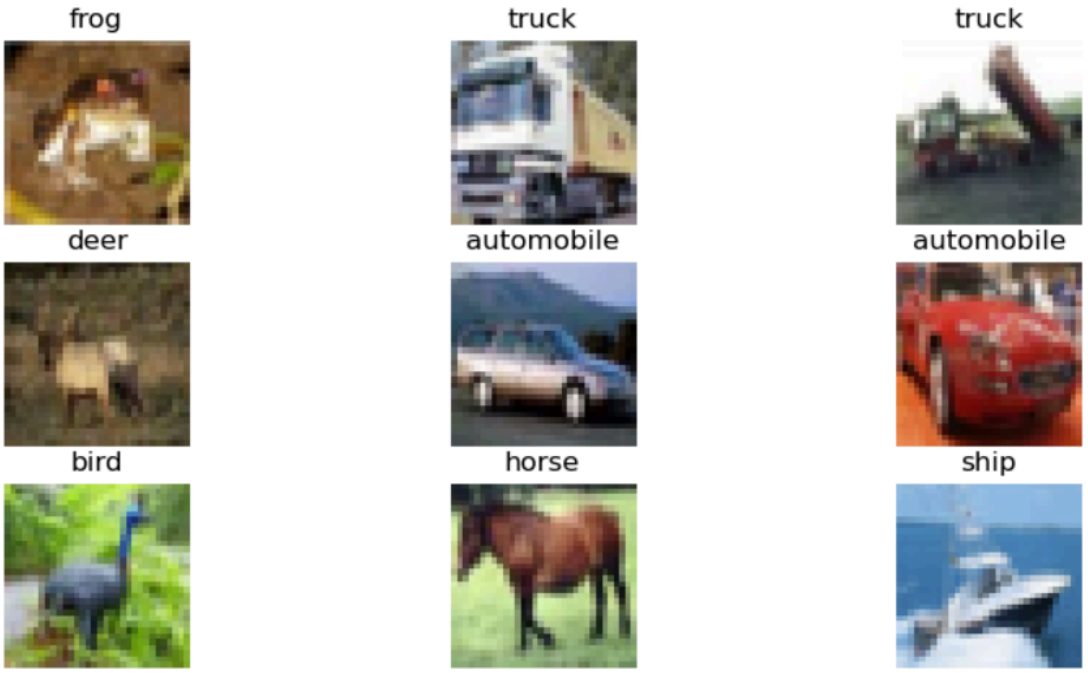

Figure 8 : Sample Images from The Dataset

The dataset contained ten object categories, including airplanes, automobiles, birds, cats, deer, dogs, frogs, horses, ships, and trucks. After normalization, all pixel values were scaled to the range of 0 to 1, which improved the stability and efficiency of the training process. The CNN model was then constructed and trained using the training dataset. The model architecture consisted of multiple convolutional layers, max-pooling layers, a flatten layer, and fully connected dense layers. Figure 9 shows the layers in the CNN model.

```
[13]: cnn.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 30, 30, 32)	896
max_pooling2d (MaxPooling2D)	(None, 15, 15, 32)	0
conv2d_1 (Conv2D)	(None, 13, 13, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 6, 6, 64)	0
flatten (Flatten)	(None, 2304)	0
dense (Dense)	(None, 64)	147,520
dense_1 (Dense)	(None, 10)	650

Total params: 167,562 (654.54 KB)
Trainable params: 167,562 (654.54 KB)
Non-trainable params: 0 (0.00 B)

Figure 9 : Layers in CNN Model

Throughout the training process, the model progressively learned important image features. As the number of epochs increased, the training accuracy improved while the training loss decreased, proving that the model was successfully learning meaningful representations from the input images. Figure 10 shows the output of the training process.

```
[17]: history = cnn.fit(
          X_train,
          y_train,
          epochs=10
      )
```

```
Epoch 1/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 28s 16ms/step - accuracy: 0.4858 - loss: 1.4398
Epoch 2/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 24s 15ms/step - accuracy: 0.6181 - loss: 1.0904
Epoch 3/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 24s 15ms/step - accuracy: 0.6637 - loss: 0.9614
Epoch 4/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 24s 16ms/step - accuracy: 0.6951 - loss: 0.8777
Epoch 5/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 24s 16ms/step - accuracy: 0.7198 - loss: 0.8080
Epoch 6/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 24s 15ms/step - accuracy: 0.7401 - loss: 0.7475
Epoch 7/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 22s 14ms/step - accuracy: 0.7577 - loss: 0.6944
Epoch 8/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 24s 15ms/step - accuracy: 0.7740 - loss: 0.6424
Epoch 9/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 22s 14ms/step - accuracy: 0.7916 - loss: 0.5954
Epoch 10/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 24s 15ms/step - accuracy: 0.8047 - loss: 0.5540
```

Figure 10 : Output of Training Process Using CNN Model

Upon evaluation using the test dataset, the new_CNN model achieved a significantly higher classification accuracy compared to the present CNN model presented in the tutorial. The CNN model achieved approximately 48.58% accuracy, whereas the new_CNN model achieved approximately 69.27% accuracy. Figure 11 shows the output for testing which shows the accuracy and loss throughout the process.

```
[19]: cnn.evaluate(X_test,y_test)

      313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - accuracy: 0.6927 - loss: 0.9593
[19]: [0.9592545628547668, 0.6927000284194946]
```

Figure 11 : Output for Testing Data

This improvement demonstrates the effectiveness of convolutional layers in capturing spatial relationships within images, which cannot be efficiently handled by traditional fully connected neural networks.

The accuracy and loss curves further provided insights into the model's learning behavior. Although the new_CNN model achieved satisfactory performance, a gap between training and validation performance could still be observed, suggesting the presence of some overfitting. This is expected because the model architecture is relatively simple and does not incorporate regularization techniques such as dropout or batch normalization. Several prediction samples were also examined to assess the model's classification capability. The new_CNN model successfully identified many objects correctly. However, some misclassifications occurred between visually similar categories such as cats and dogs or automobiles and trucks. These errors highlight the challenges associated with classifying low-resolution images in the CIFAR-10 dataset.

Overall, the results demonstrate that CNN is highly effective for image classification tasks. While the achieved accuracy of approximately 69.27% indicates good performance, further enhancements such as deeper architectures, data augmentation, dropout layers, and transfer learning techniques could be implemented to achieve even higher accuracy and better generalization.

i. Reflection

- **NABIL AFLAH BOO**

  Throughout the tutorial, the whole idea of building and checking a CNN for image classification with the CIFAR-10 dataset gave a deeper, more practical feel for how these deep learning models work beyond just theory. Working hands-on by assembling each layer of the CNN structure, doing the dataset preprocessing, and then evaluating how the accuracy and loss metrics moved across training epochs made feature extraction and pattern recognition feel a lot more real. Furthermore, after running the baseline CNN limits, like overfitting and moderate test accuracy, it shows why careful evaluation matters and enhancement methods are often needed in order to make image classification systems more solid and dependable.

- **HARINI**

  This project provided valuable experience in implementing and evaluating deep learning models for image classification. Through the comparison between the present CNN model and the new_CNN model, I learned how the new_CNN model is more effective at extracting spatial features from images, resulting in significantly higher classification accuracy. The project also enhanced my understanding of key concepts such as convolutional layers, pooling layers, activation functions, and model evaluation. Overall, this project strengthened my knowledge of deep learning and highlighted the importance of selecting an appropriate model architecture for image classification tasks.